

GraphOS

Ein Graph-natives Betriebssystem

Architektur, Implementierung und Vergleich mit
konventionellen Betriebssystemen

Lehrbuch zur GraphOS-Codebasis

Version 0.1 · Mai 2026

Geschrieben in Rust · UEFI-nativer Bootloader · `no_std`

Inhaltsverzeichnis

1	Einleitung	4
1.1	Was ist GraphOS?	4
1.2	Warum ein Graph-natives OS?	4
1.3	Historische Vorläufer	4
2	Architektur-Überblick	6
2.1	Modulstruktur	6
2.2	Schichtenmodell	6
2.3	Boot-Sequenz	7
3	Der Graph-Kernel: graphos-core	8
3.1	NodeId — Globale Identität	8
3.1.1	Vergleich mit konventionellen Systemen	8
3.2	NodeHeader — Cache-Line-Design	8
3.2.1	Node-Flags	9
3.3	Edge — Die Beziehung als Erstklassiges Objekt	9
3.3.1	EdgeKind — Semantik der Verbindung	9
3.3.2	Traversierung als universelle Operation	10
3.4	GraphPool — Die zentrale Datenstruktur	10
3.4.1	Initialisierung am Raw-Pointer	11
4	Capability-basierte Sicherheit	12
4.1	Das Problem mit konventioneller Zugriffskontrolle	12
4.2	Capabilities in GraphOS	12
4.2.1	Delegierung und Abschwächung	12
4.2.2	Vergleich: seL4 vs. GraphOS	13
4.2.3	Capability überall	13
5	Speicherverwaltung: graphos-mm	14
5.1	Buddy-Allocator	14
5.1.1	Split und Merge	14
5.1.2	Free-List-Struktur	14
5.2	Page-Table-Management	14
5.3	Frame-Allocator	15
6	IPC: Inter-Process Communication	16
6.1	Message-Design	16
6.1.1	MessageType	16
6.2	Mailbox und Channel	16
7	Content-Addressed Storage: graphos-cas	17
7.1	Das Konzept	17
7.1.1	ContentStore	17
7.1.2	Warum CAS für ein OS?	17
8	Snapshots und Live-Hotswap	18
8.1	Graph-Snapshots	18
8.1.1	Rollback	18
8.2	Hot-Swap: Live-Code-Austausch	18
8.2.1	Epoch-basiertes Concurrency-Control	18
9	Der Vektorraum: graphos-vector	20

9.1	Embeddings	20
9.1.1	Cosine-Ähnlichkeit	20
9.2	K-Nearest-Neighbors (KNN)	20
9.3	Physik-Simulation: Semantische Gravitation	21
9.4	DBSCAN-Clustering	21
10	Die Graph-Shell: graphos-shell	22
10.1	Konzept: Navigation als Graph-Traversierung	22
10.2	Pfad-Auflösung	22
10.2.1	Shell-Session	22
10.3	Natural Language Interface (Stub)	23
11	Vergleich: GraphOS vs. konventionelle Systeme	24
11.1	Fundamentale Unterschiede	24
11.2	Was GraphOS besser macht	24
11.3	Was GraphOS (noch) schlechter macht	24
12	Implementierungsdetails: Rust im Kernel	26
12.1	no_std und no_main	26
12.1.1	Globaler Allocator	26
12.2	Atomics statt Mutex	26
12.3	Serde ohne std	26
12.4	Compile-Time-Assertions	27
13	Deployment und Build	28
13.1	Rust-Toolchain	28
13.2	Build	28
13.3	QEMU-Deployment	28
13.4	Cargo-Workspace-Konfiguration	28
14	Ausblick: Was fehlt noch?	30
14.1	Kurz-/Mittelfristig	30
14.2	Langfristig	30
15	Glossar	31
16	Literatur und Referenzen	32

1 Einleitung

1.1 Was ist GraphOS?

GraphOS ist ein experimentelles Betriebssystem, das auf einem fundamentalen Paradigmenwechsel basiert: Der *gesamte* Systemzustand — Prozesse, Dateien, Geräte, Dienste, sogar KI-Modelle — wird nicht als hierarchische Dateibaum-Struktur oder als Prozessliste repräsentiert, sondern als ein einziger, lebendiger *gerichteter Graph*.

Wo ein konventionelles Betriebssystem sagt „hier ist eine Datei, dort ist ein Prozess, da ist ein Gerät,, sagt GraphOS: „alles ist ein Knoten, jede Beziehung ist eine Kante“.

Diese Idee ist nicht nur philosophisch anders — sie hat tiefgreifende praktische Konsequenzen für Sicherheit, Persistenz, Scheduling und die Art, wie ein Benutzer mit dem System interagiert.

1.2 Warum ein Graph-natives OS?

Traditionelle Betriebssysteme haben drei konzeptionell getrennte Namensräume:

- **Dateisystem:** Hierarchische Verzeichnisstruktur (Inodes, Pfade)
- **Prozessliste:** Flache Tabelle von PIDs
- **Geräte:** Separate Abstraktion (Gerätedateien unter `/dev`)

Diese Trennung ist historisch gewachsen, nicht konzeptionell notwendig. In modernen Systemen entstehen ständig Abhängigkeiten *zwischen* diesen Welten: Ein Prozess öffnet eine Datei, ein Dienst schreibt auf ein Gerät, ein Container isoliert eine Teilmenge aller drei. Jede dieser Beziehungen wird in Unix-Systemen durch separate, nicht-einheitliche Mechanismen verwaltet.

GraphOS beseitigt diese Trennung. Jedes Objekt — ob Prozess, Datei, Gerät oder KI-Modell — ist ein `Node`. Jede Beziehung ist eine `Edge`. Berechtigungen, Traversierung, Persistenz und Kommunikation laufen über dieselbe primitive Schicht.

1.3 Historische Vorläufer

GraphOS steht in einer Tradition von Systemen, die von der Hierarchie-Metapher abgewichen sind:

Plan 9 (Bell Labs, 1992) — „Alles ist eine Datei,, einheitlicher Namensraum über 9P-Protokoll. Revolutionär, aber immer noch hierarchisch.

CapOS / KeyKOS (1980er) — Capability-basierte Systeme, in denen Rechte als übertragbare Tokens durch das System fließen. GraphOS übernimmt diese Idee direkt in `CapabilityToken`.

seL4 (NICTA, 2009) — Formal verifizierbarer Mikrokern mit Capability-Space als zentralem Sicherheitsmechanismus. Inspiriert GraphOS' `CapRights`-Struktur.

Fuchsia/Zircon (Google, 2016–) — Capability-basierter Kern, Objekte als Handles. Kein Graph, aber verwandter Ansatz.

Semantic File System (MIT, 1992) — Dateien mit strukturierten Attributen statt Pfaden. Vorläufer der semantischen Navigation.

GraphOS geht weiter als alle Vorläufer: Es macht den Graph zur *einzig*en Wahrheit, ohne Fallback auf Hierarchien.

2 Architektur-Überblick

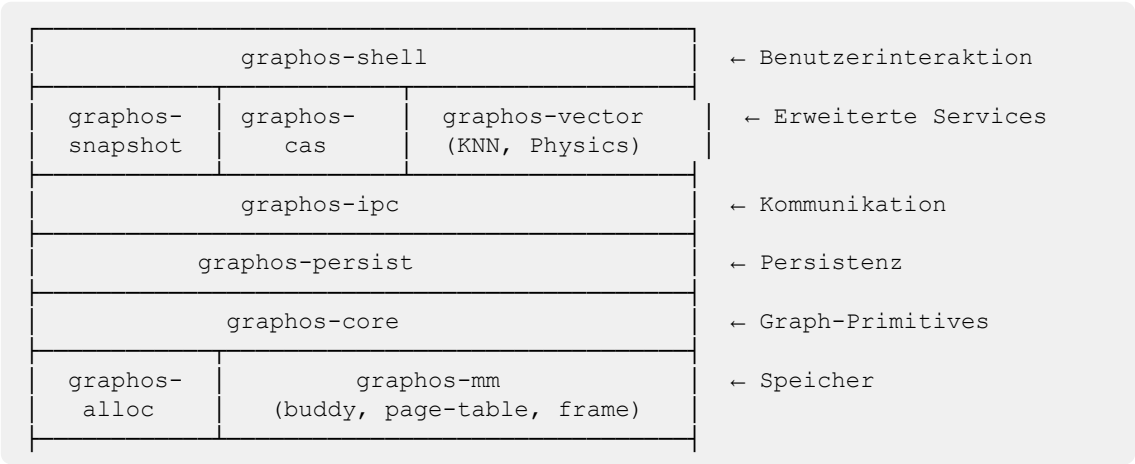
2.1 Modulstruktur

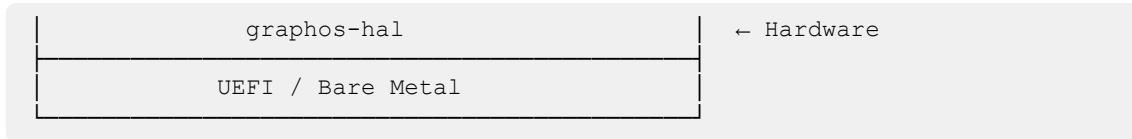
GraphOS ist als Rust-Workspace mit dreizehn Crates organisiert:

Crate	Funktion
graphos-boot	UEFI-Bootloader, System-Initialisierung, interaktive Shell
graphos-kernel	Kernel-Stub (noch minimal)
graphos-core	Fundamentale Graph-Datenstrukturen: Node, Edge, GraphPool, Capabilities
graphos-hal	Hardware Abstraction Layer (Platzhalter)
graphos-mm	Speicherverwaltung: Buddy-Allocator, Page-Tables, Frame-Tracking
graphos-alloc	Kernel-Heap-Allocator (globaler Allocator für no_std)
graphos-ipc	Inter-Process-Communication: Messages, Mailboxen, Channels
graphos-cas	Content-Addressed Storage: Blake3-Hashing, deduplizierter Content-Store
graphos-persist	Persistenzschicht: Block-Device, Serialisierung, Sektoren
graphos-snapshot	Graph-Snapshots: Erstellen, Vergleichen (Diff), Wiederherstellen
graphos-hotswap	Live-Code-Austausch: Epoch-basierter Edge-Swap ohne Neustart
graphos-vector	Vektorraummodul: Embeddings, KNN-Suche, Physics-Simulation, DBSCAN
graphos-shell	Interaktive Graph-Shell: REPL, Parser, Renderer, Session

Tabelle 1: GraphOS Crate-Übersicht

2.2 Schichtenmodell





Jede Schicht ist `#![no_std]` — sie läuft ohne die Rust-Standardbibliothek und damit direkt auf der Hardware.

2.3 Boot-Sequenz

Der Boot-Prozess in `graphos-boot/src/main.rs` ist in vier Phasen gegliedert:

Phase 1 — Graph-Initialisierung: Der `GraphPool` wird im UEFI-Speicher allokiert. Zehn System-Knoten werden erzeugt (ROOT, SCHEDULER, MEMORY, SERIAL, DISK, NETWORK, FILESYSTEM, JIT, WASM, AI_ENGINE) und durch typisierte Kanten verbunden.

Phase 2 — Subsystem-Meldungen: Alle Subsysteme (Persistenz, Memory Manager, IPC, CAS, VectorSpace) melden ihre Bereitschaft.

Phase 3 — Shell-Session: Eine interaktive `ShellSession` wird erstellt, die ROOT- und SERIAL-Knoten als Kontextpunkte benutzt.

Phase 4 — Keyboard-Loop: Das System pollt den UEFI `SimpleTextInput`-Protokoll-Zeiger und verarbeitet Tastatureingaben Zeichen für Zeichen.

3 Der Graph-Kernel: graphos-core

3.1 NodeId — Globale Identität

Jeder Knoten im System trägt eine 128-Bit-ID:

```
pub struct NodeId(pub u128);  
// Layout: [48-bit timestamp_ms | 16-bit node_type | 64-bit counter]
```

Diese Struktur enthält drei Informationen in einer einzigen Zahl:

- **Timestamp** (48 Bit): Wann wurde der Knoten erzeugt? Gibt zeitliche Ordnung.
- **Typ** (16 Bit): Welche Art von Objekt ist das? (Process, File, Device, AI_Engine, ...)
- **Zähler** (64 Bit): Atomarer, globaler Zähler — garantiert Eindeutigkeit.

Der Typ ist direkt in der ID kodiert, was type-safe Filterung *ohne* das Nachlesen des eigentlichen Node-Headers ermöglicht.

```
let nt = id.node_type(); // u16 aus Bits 64..79  
let ts = id.timestamp_ms(); // u64 aus Bits 80..127  
let ctr = id.counter(); // u64 aus Bits 0..63
```

3.1.1 Vergleich mit konventionellen Systemen

Konzept	Unix/Linux	GraphOS
Prozess-ID	PID (32-Bit-Zähler)	NodeId (128 Bit, typisiert)
Inode	64-Bit-Zahl pro Dateisystem	NodeId, global eindeutig
Geräte-ID	major:minor-Paar	NodeId mit Typ <code>Device</code>
Thread-ID	TID	NodeId mit Typ <code>Process</code>

Tabelle 2: Identitätskonzepte im Vergleich

In Unix sind PIDs, Inodes und Device-IDs konzeptionell getrennte Namensräume. In GraphOS gibt es *einen* Namensraum für alle Objekte.

3.2 NodeHeader — Cache-Line-Design

```
#[repr(C, align(64))]  
pub struct NodeHeader {  
    pub id: NodeId, // 16 Bytes  
    pub type_and_flags: AtomicU32, // 4 Bytes  
    pub refcount: AtomicU32, // 4 Bytes  
    pub edges_ptr: AtomicU64, // 8 Bytes  
    pub payload_ptr: AtomicU64, // 8 Bytes  
    pub access_cap: CapabilityToken, // 8 Bytes  
    pub region_size: u32, // 4 Bytes  
    pub edge_count: AtomicU32, // 4 Bytes  
    pub slab_index: u32, // 4 Bytes  
    _pad: [u8; 2], // 2 Bytes  
} // = 64 Bytes
```


64 Bytes entsprechen exakt einer Cache-Line auf ARM64 und x86_64. Ein Array von `NodeHeader`-Objekten liegt damit perfekt im L1-Cache — kein false sharing, keine Cache-Miss-Kaskaden.

Alle veränderlichen Felder sind `AtomicU32` / `AtomicU64`. Das ermöglicht lock-freies Lesen von mehreren Cores gleichzeitig, ohne Mutex.

Die Compile-Time-Assertion verifiziert die Größe statisch:

```
const _: () = assert!(core::mem::size_of::<NodeHeader>() == 64);
```

3.2.1 Node-Flags

Jeder Knoten trägt ein 16-Bit-Flagset, das im oberen Halbwort von `type_and_flags` gespeichert wird:

```
pub struct NodeFlags: u16 {
    const PINNED      = 0b0000_0001; // nicht GC-freigeben
    const DIRTY       = 0b0000_0010; // geänderter, ungespeicherter Zustand
    const LOCKED      = 0b0000_0100; // exklusiver Zugriff
    const GC_MARK     = 0b0000_1000; // vom GC markiert
    const PERSISTED   = 0b0001_0000; // auf Disk geschrieben
}
```

3.3 Edge — Die Beziehung als Erstklassiges Objekt

```
#[repr(C, align(64))]
pub struct Edge {
    pub source:      NodeId,          // 16 Bytes
    pub target:      NodeId,          // 16 Bytes
    pub kind:        EdgeKind,        // 1 Byte
    pub weight:      EdgeWeight,      // 4 Bytes
    pub required_cap: CapabilityToken, // 8 Bytes
    pub payload:     u64,              // 8 Bytes
    pub flags:       EdgeFlags,        // 1 Byte
    pub _pad:        [u8; 7],         // 7 Bytes
}                                     // = 64 Bytes (inklusive padding)
```

Wieder: Eine Cache-Line.

3.3.1 EdgeKind — Semantik der Verbindung

GraphOS unterscheidet 13 Kantentypen:

```
pub enum EdgeKind {
    Reference      = 0, // reine Referenz (wie ein Zeiger)
    StaticFn       = 1, // statischer Funktionsaufruf (payload = fn-ptr)
    JitCompiled    = 2, // JIT-kompilierter Code (Cranelift)
    WasmCall       = 3, // WebAssembly-Funktionsaufruf
    InferenceCall  = 4, // KI-Inferenz (LLM-Aufruf)
    Message        = 5, // asynchrone Nachricht (IPC)
    CapDelegate    = 6, // Capability-Delegierung
    Alias          = 7, // symbolischer Alias (wie Symlink)
    Stdin          = 8, // Standard-Eingabe
    Stdout         = 9, // Standard-Ausgabe
    CwdEdge        = 10, // „current working node“ der Shell
    PipeSegment    = 11, // Teil einer Pipe
}
```

```

HistoryLink = 12, // Verlaufsverbindung
}

```

Der entscheidende Unterschied zu Unix: In Unix sind ein Symlink, ein Pipe-Filedescriptor, ein Socket und ein Funktionsaufruf völlig verschiedene Konzepte mit völlig verschiedenen APIs. In GraphOS sind alle diese Beziehungen *Kanten mit verschiedenem `kind`*. Die Traversierung eines Graphen ist immer dieselbe Operation — nur die Semantik des `payload`-Feldes variiert je nach `kind`.

3.3.2 Traversierung als universelle Operation

```

pub fn traverse(&self, source: NodeId, target: NodeId, cap:
&CapabilityToken)
-> Result<TraversalResult, GraphError>
{
    let edge = self.edges[..].iter()
        .find(|e| e.source == source && e.target == target)
        .ok_or(GraphError::EdgeNotFound)?;

    if !cap.satisfies(&edge.required_cap) {
        return Err(GraphError::InsufficientRights);
    }

    match edge.kind {
        EdgeKind::Reference => Ok(TraversalResult::NodeReached(target)),
        EdgeKind::StaticFn  => Ok(TraversalResult::FnExecuted { ... }),
        _                   => Ok(TraversalResult::NodeReached(target)),
    }
}

```

Jeder Traversierungsaufruf prüft die Capability, bevor er ausgeführt wird. Sicherheit ist damit *intrinsisch* in die Graphstruktur eingebaut, nicht ein nachträglicher Filter.

3.4 GraphPool — Die zentrale Datenstruktur

```

pub struct GraphPool {
    nodes:      &'static mut [NodeHeader],
    edges:      &'static mut [Edge],
    node_count: usize,
    edge_count: usize,
    id_gen:     NodeIdGenerator,
    config:     GraphPoolConfig,
}

```

Der `GraphPool` verwendet zwei zusammenhängende Arrays für Knoten und Kanten. Dieses *flat array*-Design hat mehrere Vorteile gegenüber pointer-verlinkten Strukturen:

1. **Cache-Effizienz:** Lineare Iteration durch alle Knoten/Kanten ist vorhersehbar für den CPU-Prefetcher.
2. **Kein Heap-Overhead:** Kein dynamisches Allokieren pro Knoten.
3. **Serialisierbarkeit:** Ein `memcpy` genügt für Snapshots.
4. **NUMA-freundlich:** Der gesamte Pool kann auf einem NUMA-Knoten alloziert werden.

Der Nachteil: Suche nach einem einzelnen Knoten ist $O(n)$. Das aktuelle Design arbeitet mit einem linearen Scan — kommentiert als „wird durch HashMap ersetzt“.

3.4.1 Initialisierung am Raw-Pointer

```
pub unsafe fn init_at(base: *mut u8, config: GraphPoolConfig)
-> &'static mut Self
```

Der Pool wird direkt in einem vorab allokierten Speicherbereich platziert. UEFI liefert diesen Speicher. Es gibt kein `malloc`, kein Heap — der gesamte Pool-Zustand liegt in einem zusammenhängenden Block.

4 Capability-basierte Sicherheit

4.1 Das Problem mit konventioneller Zugriffskontrolle

Unix-Systeme verwenden *discretionary access control* (DAC): Dateien haben Besitzer und Berechtigungsbits. Wer als Prozess läuft, erbt die Rechte des Benutzers. Das führt zu fundamentalen Problemen:

- **Confused Deputy Problem:** Ein privilegierter Prozess (z.B. ein Webserver) kann von einem weniger privilegierten Aufrufer dazu gebracht werden, Dinge in seinem Namen zu tun, für die der Aufrufer selbst keine Rechte hätte.
- **Ambient Authority:** Prozesse haben alle Rechte des Benutzers, auch wenn sie nur einen Bruchteil davon benötigen (Principle of Least Privilege verletzt).
- **Kein revokierbarer Zugriff:** Einmal geöffnete Filedescriptoren können nicht selektiv entzogen werden.

4.2 Capabilities in GraphOS

```
#[repr(C)]
pub struct CapabilityToken {
    pub rights: CapRights, // 2 Bytes: was darf ich?
    pub scope:  u16,       // 2 Bytes: auf welche Knoten-Typen?
    pub badge:  u32,       // 4 Bytes: wer bin ich?
}                               // = 8 Bytes
```

Ein `CapabilityToken` ist ein *unvergälschbares, miniaturisiertes Zugriffsrecht*. Es kodiert:

- **rights:** Bitmaske der erlaubten Operationen (READ, WRITE, EXECUTE, TRAVERSE, CREATE, DELETE, DELEGATE, REVOKE, GRANT, KERNEL)
- **scope:** Auf welche Node-Typen gilt dieses Token? (`0xFFFF` = universell)
- **badge:** Wer hat dieses Token erzeugt/delegiert?

```
pub struct CapRights: u16 {
    const READ      = 0b0000_0000_0001;
    const WRITE     = 0b0000_0000_0010;
    const EXECUTE   = 0b0000_0000_0100;
    const TRAVERSE  = 0b0000_0000_1000;
    const CREATE    = 0b0000_0001_0000;
    const DELETE    = 0b0000_0010_0000;
    const DELEGATE  = 0b0000_0100_0000;
    const REVOKE    = 0b0000_1000_0000;
    const GRANT     = 0b0001_0000_0000;
    const KERNEL    = 0b1000_0000_0000;
}
```

4.2.1 Delegierung und Abschwächung

```
pub fn derive(&self, mask: CapRights, new_scope: u16) -> Option<Self> {
    if !self.rights.contains(CapRights::DELEGATE) {
        return None;
    }
    let restricted_rights = self.rights & mask;
    Some(Self {
        rights: restricted_rights,
```

```

        scope: new_scope,
        badge: self.badge,
    })
}

```

Ein Token kann nur *abgeschwächt* werden, nie verstärkt: `restricted_rights = self.rights & mask`. Wer `DELEGATE` hat, kann ein Token ableiten, das eine Teilmenge seiner eigenen Rechte enthält. Diese Eigenschaft (Monotonie der Abschwächung) ist das Kernprinzip von Capability-Systemen nach Saltzer & Schroeder.

4.2.2 Vergleich: seL4 vs. GraphOS

Eigenschaft	seL4	GraphOS
Granularität	Objekt-Handles	NodeId-spezifisch
Übertragung	IPC-Kanal	<code>derive()</code> -Methode
Revokation	Cspace-Hierarchie (geplant)	<code>REVOKE</code> -Recht
Größe des Tokens	64-Bit-Handle	64-Bit-Struct
Scope	typenlos	<code>scope: u16</code> (Knotentyp-Filter)

Tabelle 3: Capability-Systeme im Vergleich

GraphOS vereinfacht seL4s Capability-Space-Konzept auf eine flache, direkt in jede Datenstruktur einbettbare Form.

4.2.3 Capability überall

Entscheidend: Jede Kante trägt ein `required_cap`-Feld. Jeder Knoten trägt `access_cap`. Jede Operation nimmt eine Capability als Parameter. Es gibt keinen einzigen Codepfad, der Zugriff ohne Capability-Prüfung gewährt:

```

// Knoten allokkieren erfordert CREATE
if !cap.rights.contains(CapRights::CREATE) {
    return Err(GraphError::InsufficientRights);
}

// Kante verbinden erfordert WRITE
if !cap.rights.contains(CapRights::WRITE) {
    return Err(GraphError::InsufficientRights);
}

// Traversierung erfordert TRAVERSE + passende required_cap der Kante
if !cap.satisfies(&edge.required_cap) {
    return Err(GraphError::InsufficientRights);
}

```

5 Speicherverwaltung: graphos-mm

5.1 Buddy-Allocator

Der klassische Buddy-Allocator teilt den physischen Speicher in Zweierpotenzen auf. GraphOS implementiert ihn für Order 0 (4 KiB) bis Order 15 (128 MiB):

```
Order 0: 4 KiB   (1 Page)
Order 1: 8 KiB   (2 Pages)
Order 2: 16 KiB  (4 Pages)
...
Order 15: 128 MiB (32768 Pages)
```

5.1.1 Split und Merge

Ist kein Block der gewünschten Größe verfügbar, wird der nächstgrößere Block aufgeteilt:

```
Order-4-Block (64 KiB) wird gesplittet in:
→ linker Buddy (32 KiB, Order 3) → in Freiliste eingefügt
→ rechter Buddy (32 KiB, Order 3) → weiter splitten bis Order 1
```

Beim Freigeben wird der Buddy-Adresse mit dem XOR-Trick berechnet:

```
let buddy_relative = relative ^ block_size;
```

Das funktioniert, weil Buddy-Paare immer an Adressen liegen, die sich nur im bit für ihre Ordnung unterscheiden.

5.1.2 Free-List-Struktur

```
pub struct BuddyAllocator {
    free_lists: [Option<*mut FreeBlock>; 16],
    base: usize,
    total_size: usize,
}
```

Jede Ordnung hat ihre eigene Freiliste als einfach verkettete Liste von `FreeBlock`-Structs, die *im freien Speicher selbst* liegen (in-place metadata). Das ist möglich, weil der Speicher zu dem Zeitpunkt, wo er in der Freiliste ist, nicht anderweitig benutzt wird.

5.2 Page-Table-Management

Virtuelles Layout:

Kernel-Mapping	0xFFFF_FFFF_FFFF_FFFF (higher half, PML4[511])
...	0xFFFF_8000_0000_0000
User-Space	0x0000_7FFF_FFFF_FFFF (lower half)
	0x0000_0000_0000_0000

Der `x86_64` 4-stufige Page-Walk (`PML4 → PDPT → PD → PT`) ist in `graphos-mm/src/page_table.rs` implementiert, mit Identity-Mapping für physisch-virtuelle Gleichheit im frühen Boot.

5.3 Frame-Allocator

Oberhalb des Buddy-Allocators verwaltet ein Bitmap-basierter Frame-Allocator physische Seitenrahmen:

```
pub struct FrameAllocator {  
    bitmap: &'static mut [u64],  
    total_frames: usize,  
    free_frames: usize,  
    base_frame: usize,  
}
```

1 Bit pro Frame (0 = frei, 1 = belegt). 1024 Frames = 4 MiB werden mit einem einzigen `u64`-Array von 16 Einträgen verwaltet.

6 IPC: Inter-Process Communication

6.1 Message-Design

```
#[repr(C, align(64))]
pub struct Message {
    pub sender:      NodeId,      // 16 Bytes
    pub receiver:    NodeId,      // 16 Bytes
    pub msg_type:     MessageType, // 2 Bytes
    pub payload_len: u16,         // 2 Bytes
    pub timestamp:   u32,         // 4 Bytes
    pub payload:     [u8; 24],    // 24 Bytes
                                // = 64 Bytes
}
```

Wieder: Eine Cache-Line pro Message. Der Payload ist auf 24 Bytes beschränkt (inline). Für größere Daten: eine Referenz auf einen `ContentAddr`-Knoten im CAS.

Sender und Empfänger sind `NodeIds` — nicht Prozess-IDs, Socket-Handles oder File-Descriptors. Jeder Knoten kann Nachrichten senden und empfangen, sofern er die entsprechenden Capabilities hat.

6.1.1 MessageType

```
pub enum MessageType {
    Data      = 0x0001, // rohe Daten
    Signal     = 0x0002, // Signal (wie Unix-SIGTERM)
    Request    = 0x0003, // RPC-Anfrage
    Reply      = 0x0004, // RPC-Antwort
    Error      = 0x0005, // Fehler
    GraphEvent = 0x0010, // Kante hinzugefügt/entfernt/modifiziert
}
```

`GraphEvent` ist besonders interessant: Das System selbst kann Nachrichten senden, wenn sich der Graph ändert. Das ermöglicht reaktive Programmierung auf Kernel-Ebene — ein Dienst kann sagen „benachrichtige mich, wenn eine neue Kante von diesem Knoten ausgeht“.

6.2 Mailbox und Channel

```
pub struct Mailbox<const N: usize> {
    buf: [Message; N],
    head: AtomicU32,
    tail: AtomicU32,
}
```

Ein lock-freier Ring-Buffer als SPSC-Queue (Single Producer, Single Consumer). Die `AtomicU32`-Köpfe erlauben race-freies Lesen ohne Mutex.

Ein `Channel` koppelt zwei Mailboxen ($A \rightarrow B$, $B \rightarrow A$) zu einer bidirektionalen Verbindung. Eine `ChannelRegistry` verwaltet bis zu 16 Channels, adressierbar über `ChannelId`.

7 Content-Addressed Storage: graphos-cas

7.1 Das Konzept

In konventionellen Dateisystemen werden Dateien durch ihren *Namen* (Pfad) adressiert. Zwei identische Dateien an verschiedenen Orten belegen doppelten Speicher. Content-Addressed Storage adressiert Daten durch ihren *Inhalt* (Hash). Identische Daten werden automatisch dedupliziert.

GraphOS übernimmt dieses Konzept aus Git, IPFS und Nix. Jedes Datum bekommt eine `ContentId`, berechnet als 256-Bit-Hash des Inhalts:

```
pub struct ContentId {
    pub hash: Hash256, // Blake3-artiger 256-Bit-Hash
}

pub struct Hash256 {
    pub bytes: [u8; 32],
}
```

7.1.1 ContentStore

```
pub struct ContentStore {
    entries: [ContentEntry; 256],
    count:    usize,
}

pub struct ContentEntry {
    pub id:    ContentId,
    pub data:  [u8; 128], // Inline für kleine Objekte
    pub len:   u32,
    pub flags: u8,
}
```

Für kleine Objekte (≤ 128 Bytes) speichert der Store die Daten direkt inline. Für größere Objekte würde ein Verweis auf den Block-Device-Layer eingefügt. Der Store bietet `put`, `get` und `contains` — eine minimalistische Key-Value-Datenbank, deren Schlüssel der Inhalt selbst bestimmt.

7.1.2 Warum CAS für ein OS?

- **Snapshots ohne Kopien:** Zwei Snapshots teilen automatisch alle identischen Blöcke.
- **Integritätsprüfung:** Hash-Verifikation ist trivial.
- **Versionierung:** Jede Version eines Knotens ist eine separate `ContentId`.
- **Deduplication:** Identische Kernel-Module, die mehrfach geladen werden, belegen nur einmal Speicher.

8 Snapshots und Live-Hotswap

8.1 Graph-Snapshots

Ein Snapshot ist eine vollständige Momentaufnahme des Graph-Zustands:

```
pub struct SnapshotHeader {
    pub magic:      [u8; 8],    // "GRAPHOS\0"
    pub version:    u32,
    pub epoch:      u64,        // logische Zeit
    pub node_count: u32,
    pub edge_count: u32,
    pub checksum:   u64,        // FNV1a-64 des gesamten Datums
    pub flags:      u32,
}
```

`SnapshotDiff` berechnet die Differenz zwischen zwei Snapshots:

```
pub struct SnapshotDiff {
    pub added_nodes:    usize,
    pub removed_nodes:  usize,
    pub added_edges:    usize,
    pub removed_edges:  usize,
    pub modified_nodes:  usize,
}
```

Das ermöglicht *inkrementelle Persistenz*: Nur die geänderten Knoten und Kanten müssen auf Disk geschrieben werden, nicht der gesamte Graph.

8.1.1 Rollback

```
pub fn restore_snapshot(pool: &mut GraphPool, header: &SnapshotHeader,
    node_data: &[u8], edge_data: &[u8]) -> Result<(), SnapshotError>
```

Der `GraphPool` kann auf einen vorherigen Zustand zurückgesetzt werden. Da Knoten und Kanten in flat arrays liegen, ist Restore ein `memcpy` plus Zähler-Reset.

8.2 Hot-Swap: Live-Code-Austausch

Das Hotswap-Modul ermöglicht, eine Kante im laufenden Betrieb auszutauschen — z.B. um eine JIT-kompilierte Funktion durch eine neue Version zu ersetzen, ohne das System neu zu starten.

8.2.1 Epoch-basiertes Concurrency-Control

```
pub struct EpochTracker {
    current_epoch: AtomicU64,
    active_readers: [AtomicU32; MAX_EPOCHS],
}

pub struct EpochGuard<'a> {
    tracker: &'a EpochTracker,
    epoch: u64,
}
```

Das Epoch-Konzept (aus der RCU-Literatur bekannt) funktioniert so:

1. Jeder Leser registriert sich bei einem Epoch.
2. Ein Swap erhöht die Epoch.
3. Erst wenn die alte Epoch keine aktiven Leser mehr hat, kann der alte Code freigegeben werden.

```
pub struct SwapRecord {  
    pub old_payload:    u64,  
    pub new_payload:    u64,  
    pub swap_epoch:     u64,  
    pub source:         NodeId,  
    pub target:         NodeId,  
}
```

`hot_swap_edge` atomisch tauscht das `payload`-Feld einer Kante aus und protokolliert den Austausch im `SwapRecord`. Kein Neustart erforderlich.

9 Der Vektorraum: graphos-vector

Das Vektorraummodul ist die „KI-native,, Schicht von GraphOS — das, was es von allen anderen Betriebssystemen unterscheidet.

9.1 Embeddings

```
pub struct EmbeddingVector {  
    pub dims: [f32; 64], // 64-dimensionaler Vektor  
}
```

Jeder Knoten im System kann einen 64-dimensionalen Embedding-Vektor tragen. Dieser Vektor kodiert die *semantische Bedeutung* des Knotens in einem kontinuierlichen Raum.

Im aktuellen Stand werden Embeddings aus einem Seed deterministisch generiert (für Tests). In einer produktiven Version würde ein eingebettetes LLM (z.B. TinyLlama 1.1B Q4, bereits als Node `AI_ENGINE` registriert) echte Embeddings erzeugen.

9.1.1 Cosine-Ähnlichkeit

```
pub fn cosine_similarity(&self, other: &Self) -> f32 {  
    let dot = self.dims.iter().zip(other.dims.iter())  
        .map(|(a, b)| a * b).sum::<f32>();  
    let norm_self = self.dims.iter().map(|x| x * x).sum::<f32>().sqrt();  
    let norm_other = other.dims.iter().map(|x| x * x).sum::<f32>().sqrt();  
    dot / (norm_self * norm_other + 1e-8)  
}
```

Weil kein `libm` in `no_std` verfügbar ist, implementiert GraphOS die Quadratwurzel mit dem berühmten Quake-III-Algorithmus (inverse square root, drei Newton-Iterationen):

```
fn sqrt_f32(x: f32) -> f32 {  
    let i = f32::to_bits(x);  
    let i = 0x5f3759df - (i >> 1);  
    let mut guess = 1.0 / f32::from_bits(i);  
    guess = 0.5 * (guess + x / guess); // Newton-Raphson  
    guess = 0.5 * (guess + x / guess);  
    guess = 0.5 * (guess + x / guess);  
    guess  
}
```

Die magische Konstante `0x5f3759df` liefert eine Startschätzung für $\frac{1}{\sqrt{x}}$, die durch Newton-Iterationen verfeinert wird.

9.2 K-Nearest-Neighbors (KNN)

```
pub fn knn(query: &EmbeddingVector, k: usize,  
    space: &VectorSpace, results: &mut [KnnResult]) -> usize
```

Linearer Scan durch alle Knoten, gemessen in Cosine-Distanz ($1 - \text{sim}$). Ein Insertion-Sort-Puffer der Größe k hält die k bisher nächsten Nachbarn. Laufzeit: $O(n \cdot d)$ für n Knoten mit d Dimensionen.

Das ermöglicht: „Finde die 5 Knoten im System, die diesem Knoten am semantisch ähnlichsten sind.“ In einem Unix-System wäre diese Frage nicht mal sinnvoll formulierbar.

9.3 Physik-Simulation: Semantische Gravitation

```
pub struct PhysicsConfig {
    pub coulomb_constant: f32, // 100.0 – abstoßende Kraft
    pub spring_constant: f32, // 0.1 – Federkraft für verbundene
    Knoten
    pub damping: f32, // 0.95 – Reibung
    pub semantic_gravity: f32, // 5.0 – Anziehung ähnlicher Knoten
    pub dt: f32, // 0.016 – Zeitschritt (60 FPS)
    pub similarity_threshold: f32, // 0.5 – ab wann gilt Ähnlichkeit?
}
```

Jeder Knoten im Vektorraum hat eine 3D-Position und eine Geschwindigkeit. Der Physik-Schritt berechnet für jeden Knoten die wirkenden Kräfte:

1. **Coulomb-Abstoßung:** Alle Knoten stoßen sich ab (wie gleichnamige Ladungen). Das verhindert, dass alle Knoten übereinander liegen.
2. **Federkraft:** Verbundene Knoten (durch Graphkanten) werden zusammengezogen.
3. **Semantische Gravitation:** Knoten mit hoher Cosine-Ähnlichkeit ihrer Embeddings ziehen sich zusätzlich an — unabhängig davon, ob eine explizite Kante existiert.

Das Ergebnis: Der Graph *arrangiert sich selbst* im 3D-Raum so, dass semantisch ähnliche Knoten räumlich nah beieinander liegen. Das ist eine Form von *emergenter Topologie* — die Struktur des Graphen ergibt sich aus den Daten.

9.4 DBSCAN-Clustering

```
pub fn dbscan(space: &VectorSpace, epsilon: f32, min_points: u32,
    results: &mut [ClusterResult]) -> usize
```

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) findet Cluster von ähnlichen Knoten:

- Knoten mit mindestens `min_points` Nachbarn im Radius `epsilon` werden zu *Kernpunkten*.
- Kernpunkte, die erreichbar sind, werden zu einem Cluster zusammengefasst.
- Knoten außerhalb aller Cluster sind *Noise*.

Das Ergebnis: Das OS erkennt automatisch, welche seiner eigenen Knoten thematisch zusammengehören — ohne explizite Konfiguration.

10 Die Graph-Shell: graphos-shell

10.1 Konzept: Navigation als Graph-Traversierung

Die Shell in einem traditionellen Unix-System navigiert eine Verzeichnishierarchie.

`cd /home/user/docs` folgt einem Pfad. `ls` listet Inhalte eines Verzeichnisses.

Die GraphOS-Shell navigiert den Graphen. `go scheduler` folgt einer Kante von `CwdEdge`-Typ. `look` zeigt alle Kanten, die vom aktuellen Knoten ausgehen.

```
pub enum Command {
    Look { deep: bool },           // zeige Knoten + Kanten
    Go { path: Path },             // bewege zum Knoten
    Spawn { type_name, name },     // erzeuge neuen Knoten
    Who,                           // zeige Session-Info
    Touch { edge_name },           // traversiere benannte Kante
    Find { type_filter, max_depth }, // BFS-Suche
    Bind { name, target },         // erzeuge Alias
    Cut { src, target },           // trenne Kante
    Link { src, target, kind },    // verbinde Knoten
    Ask { prompt },               // NLP-Anfrage (KI-Stub)
    Snapshot,                     // erstelle Snapshot
    Rollback { epoch },           // stelle Snapshot wieder her
    Store { data },               // speichere in CAS
    Similar { k },                // finde ähnliche Knoten (KNN)
    Physics { steps },            // führe Physik-Schritte aus
    Clusters,                     // zeige Cluster
    NaturalLanguage { text },     // Intent-Parsing (Stub)
}
```

10.2 Pfad-Auflösung

```
pub enum Path {
    Absolute(Vec<u64>), // /root/scheduler/jit → [hash("root"), ...]
    Relative(Vec<u64>), // scheduler/jit → [hash("scheduler"), ...]
    Parent,             // .. (noch nicht implementiert)
    Alias(String),      // @myalias → lookup in Binding-Knoten
    Direct(String),     // #0x1234 → direkte NodeId
}
```

Pfade werden als Sequenzen von FNV1a-64-Hashes der Kantennamen gespeichert.

`resolve_path` folgt diesen Hashes durch den Graphen.

10.2.1 Shell-Session

```
pub struct ShellSession {
    pub node_id: NodeId, // der Shell-Knoten selbst
    pub cwd: NodeId,     // "current working node"
    pub cap: CapabilityToken,
    pub binding_node: NodeId, // Knoten für Alias-Verwaltung
    // ...
}
```

Die Shell ist selbst ein Knoten im Graph. Ihr `cwd` (current working directory — hier: current working *node*) ist eine `CwdEdge`-Kante, die bei `go`-Befehlen umgehängt wird (`rehang_edge`).

10.3 Natural Language Interface (Stub)

```
Command::NaturalLanguage { text } => {  
    Ok(format!("[AI] Intent not yet connected: \"{}\"", text))  
}  
Command::Ask { prompt } => {  
    Ok(format!("[AI] {}", prompt))  
}
```

Die Infrastruktur für Natural Language Interface ist bereits eingebaut — der Parser erkennt freitextliche Eingaben und leitet sie an `NaturalLanguage` weiter. Der `AI_ENGINE`-Knoten (TinyLlama 1.1B Q4) würde in einer vollständigen Implementierung Intent-Parsing übernehmen: „zeig mir alle Prozesse die mehr als 100MB Speicher belegen“, → programmatisch ausgeführte Graph-Traversierung.

11 Vergleich: GraphOS vs. konventionelle Systeme

11.1 Fundamentale Unterschiede

Konzept	Unix/Linux	GraphOS
Objekt-Modell	Getrennte Namensräume (PID, Inode, FD, ...)	Einheitliche NodeId über alle Objekte
Dateisystem	Hierarchische Verzeichnisse	Graph mit benannten Kanten
Prozesse	Flache Tabelle mit PID	Knoten vom Typ <code>Process</code>
IPC	Pipes, Sockets, Signals, SHM (4+ Mechanismen)	Kanten vom Typ <code>Message / Channel</code>
Berechtigungen	UID/GID-Bits (DAC), SELinux optional (MAC)	Capability-Tokens an jeder Kante/Knoten
Persistenz	ext4/btrfs Journaling	CAS + Snapshots + Diff
Live-Update	Nicht ohne Neustart	Hot-Swap via Epoch-Guards
Semantik	Keine — alles sind Bytes	Embeddings + KNN + Physik
Shell	Pfad-Navigation im VFS	Graph-Traversierung mit Capability-Check
Code-Execution	fork/exec + ELF-Loading	<code>StaticFn</code> / <code>JitCompiled</code> / <code>WasmCall</code> -Kanten

Tabelle 4: GraphOS vs. Unix/Linux — fundamentale Unterschiede

11.2 Was GraphOS besser macht

Einheitlichkeit: In Unix hat man `open()` für Dateien, `socket()` für Netzwerk, `fork()` für Prozesse, `mmap()` für Speicher. In GraphOS gibt es `alloc_node()` + `connect()` für alles.

Sicherheit by Design: Jede Operation prüft eine Capability. Es ist strukturell unmöglich, auf einen Knoten zuzugreifen, ohne ein gültiges Token zu haben.

Persistenz: Der gesamte Systemzustand ist ein serialisierbares Array. Ein Snapshot ist konzeptionell ein `memcpy`.

Semantische Navigation: Das System kann seine eigene Struktur „verstehen“ — ähnliche Knoten finden, Cluster erkennen, Pfade über semantische Nähe traversieren.

11.3 Was GraphOS (noch) schlechter macht

Performance: Lineare Suche $O(n)$ statt $O(1)$ -Hashtabellen. Kein $O(1)$ -Lookup nach NodeId.

Fehlende POSIX-Schicht: Kein `fork`, kein `exec`, keine Datei-Deskriptoren. Bestehende Programme laufen nicht.

Kein Scheduler: Der `Scheduler`-Knoten ist registriert, aber leer implementiert. Work-Stealing-Executor ist noch kein Rust-Code.

Kein echter Treiber: Alle Geräte (DISK, NETWORK) sind Knoten ohne Backend-Implementierung.

Keine MMU-Isolation: Alle Knoten teilen denselben Adressraum. Kein Prozessschutz durch Page-Tables (noch).

12 Implementierungsdetails: Rust im Kernel

12.1 no_std und no_main

```
#![no_std]
#![no_main]
```

Beide Attribute definieren, wie GraphOS auf den Standard-Bibliotheken aufbaut (gar nicht). `no_std` entfernt die gesamte `std`-Crate; `no_main` erlaubt einen custom Einstiegs-punkt über UEFI's `#[entry]`-Makro.

Verfügbar ohne `std`:

- `core` — Primitive Typen, Iteratoren, Slices
- `alloc` — Dynamische Allokation (benötigt globalen Allocator)
- Crates mit `default-features = false`

Nicht verfügbar:

- Filesystem-Zugriff, Netzwerk, Threads, `println!`, Panics mit Backtrace

12.1.1 Globaler Allocator

```
// graphos-alloc/src/lib.rs
#[global_allocator]
static ALLOCATOR: LockedAllocator = LockedAllocator::new();
```

`graphos-alloc` implementiert den `GlobalAlloc`-Trait. Der UEFI-Allocator delegiert an `uefi::boot::allocate_pool`. Dadurch ist `alloc::vec::Vec`, `alloc::string::String` etc. nutzbar.

12.2 Atomics statt Mutex

In frühen Kernelphasen (vor dem Scheduler) gibt es keine Threads — daher auch keine echten Races. Dennoch werden `AtomicU32` / `AtomicU64` konsequent verwendet:

- Als selbst-dokumentierende Konvention: „dieses Feld kann concurrent gelesen werden,“
- Für spätere SMP-Kompatibilität ohne Code-Umstrukturierung
- Für lock-freie Algorithmen (Mailbox-Queue, Edge-Counter)

Die `Ordering`-Wahl ist durchgehend `Relaxed` — ausreichend für Single-Core UEFI, aber bewusst als Upgrade-Punkt markiert.

12.3 Serde ohne std

```
serde = { version = "1.0", default-features = false, features =
["derive"] }
postcard = { version = "1.0", default-features = false, features =
["alloc"] }
```

`postcard` ist ein kompakter Serialisierer für `no_std` — optimiert auf kleine binäre Ausgabe ohne Heap-Allokation wenn möglich. Er wird für die Persistenzschicht verwendet:

`NodeHeader` und `Edge` können direkt serialisiert und auf den Block-Device geschrieben werden.

12.4 Compile-Time-Assertions

```
const _: () = assert!(core::mem::size_of::<NodeHeader>() == 64);
const _: () = assert!(core::mem::align_of::<NodeHeader>() == 64);
const _: () = assert!(core::mem::size_of::<Edge>() == 64);
const _: () = assert!(core::mem::size_of::<Message>() == 64);
const _: () = assert!(core::mem::size_of::<CapabilityToken>() == 8);
```

Diese `const`-Assertions werden zur Compile-Zeit ausgewertet. Wenn jemand ein Feld zur `NodeHeader`-Struktur hinzufügt und dadurch die Größe auf 65 Bytes steigt, bricht der Build — kein Runtime-Fehler, keine stille Cache-Degradation.

13 Deployment und Build

13.1 Rust-Toolchain

```
# rust-toolchain.toml
[toolchain]
channel = "nightly"
targets = [
    "x86_64-unknown-uefi",
    "x86_64-unknown-none",
]
components = ["rust-src", "llvm-tools"]
```

GraphOS benötigt Nightly Rust wegen `#![feature(allocator_api)]`. Das UEFI-Target `x86_64-unknown-uefi` produziert PE32+-Binärdateien (Windows-Executable-Format), wie UEFI sie erwartet.

13.2 Build

```
cargo build --package graphos-boot \
    --target x86_64-unknown-uefi \
    --release
```

Die EFI-Datei liegt dann unter:

```
target/x86_64-unknown-uefi/release/graphos-boot.efi
```

13.3 QEMU-Deployment

```
# configs/qemu/deploy.sh
qemu-system-x86_64 \
    -enable-kvm \
    -m 512M \
    -drive if=pflash,format=raw,readonly=on,file=OVMF_CODE.fd \
    -drive if=pflash,format=raw,file=OVMF_VARS.fd \
    -drive format=raw,file=fat:rw:esp \
    -serial stdio \
    -display none
```

OVMF (Open Virtual Machine Firmware) liefert das UEFI-Environment. Die EFI-Partition wird als FAT-Verzeichnis gemountet. QEMU bootet direkt in GraphOS.

13.4 Cargo-Workspace-Konfiguration

```
[profile.release]
opt-level = 3
lto = "fat"           # Link-Time Optimization über alle Crates
codegen-units = 1     # Maximale Optimierung (single codegen unit)
panic = "abort"       # Kein Panic-Unwinding im Kernel

[profile.dev]
opt-level = 1         # Etwas Optimierung für bessere Fehlersuche
panic = "abort"
```

`panic = "abort"` ist Pflicht: Stack-Unwinding erfordert `libunwind`, das in `no_std` nicht verfügbar ist. Ein Kernel-Panic beendet das System.

14 Ausblick: Was fehlt noch?

GraphOS v0.1 ist ein funktionierender Proof-of-Concept. Der Graph lebt, die Shell ist interaktiv, die Primitiven sind korrekt implementiert. Folgendes ist noch nicht realisiert:

14.1 Kurz-/Mittelfristig

O(1)-Knotensuche — Ein Hash-Index über `NodeId → Slab-Position` würde die aktuelle O(n)-Suche in `find_node()` auf O(1) reduzieren.

Work-Stealing-Scheduler — Der `Scheduler`-Knoten hat keine Implementierung. Ein async-Runtime auf Basis von Rust's `Futures` würde parallel laufende Knoten ermöglichen.

Echter Treiber-Stack — DISK- und NETWORK-Knoten als virtio-Backend-Implementierungen.

MMU-Isolation — Verschiedene Knoten in verschiedenen Page-Table-Contexts → Prozessisolation.

14.2 Langfristig

Verteilter Graph — Knoten auf verschiedenen Maschinen, Kanten über Netzwerk (GraphQL oder custom Protokoll). Ein echtes Peer-to-Peer-OS.

Formale Verifikation der Capabilities — Proof dass kein Programm mit einem eingeschränkten Token jemals an Kernel-Privilegien kommt (ähnlich seL4's Coq-Proof).

JIT-Compiler-Integration — Cranelift als `JitCode`-Kanten-Backend. Funktionen werden zur Laufzeit kompiliert und als Kanten in den Graph eingefügt.

Vollständiges LLM-Backend — TinyLlama auf dem `AI_ENGINE`-Knoten für echtes Intent-Parsing in der Shell.

15 Glossar

Buddy-Allocator — Speicherverwaltung, die Blöcke in Zweierpotenzen aufteilt und beim Freigeben automatisch zusammenführt.

Capability — Unvergälschbares Zugriffsrecht, das Operationen (READ, WRITE, ...) und Scope (Knotentypen) kodiert. Inspiriert von seL4.

CAS (Content-Addressed Storage) — Datenspeicherung, bei der der Hash des Inhalts als Adresse dient. Automatische Deduplizierung.

Cache-Line — Minimale Speichereinheit, die der CPU-Cache lädt (64 Bytes auf ARM64/x86_64). Datenstrukturen sollten Cache-Lines ausrichten.

DBSCAN — Density-Based Spatial Clustering. Findet beliebig geformte Cluster ohne vorgegebene Cluster-Anzahl.

Embedding — Abbildung eines Objekts auf einen Vektor im kontinuierlichen Raum. Semantisch ähnliche Objekte haben ähnliche Vektoren.

Epoch — Logischer Zeitstempel für concurrency-sichere Speicherverwaltung (RCU-Konzept).

FNv1a — Fowler-Noll-Vo Hash-Funktion. Schnell, nicht-kryptographisch, gut verteilt. Benutzt für Edge-Namen-Hashes.

GraphPool — Zentrale Datenstruktur: zwei flache Arrays für Knoten und Kanten, plus Zähler.

Hot-Swap — Live-Austausch einer Kanten-Implementierung ohne Systemneustart.

KNN (K-Nearest-Neighbors) — Algorithmus zur Suche der k ähnlichsten Vektoren zu einem Query-Vektor.

NodeId — 128-Bit-Identifizier: Timestamp (48 Bit) + NodeType (16 Bit) + Zähler (64 Bit).

no_std — Rust-Compilation ohne Standardbibliothek. Notwendig für Bare-Metal-Kernel.

UEFI (Unified Extensible Firmware Interface) — Moderner BIOS-Nachfolger. Bietet Boot-Services, Speicherallokation, Keyboard/Display-APIs vor dem eigentlichen OS-Kern.

16 Literatur und Referenzen

- [1] Andrew S. Tanenbaum, Herbert Bos: *Modern Operating Systems*, 4th Edition. Pearson, 2014.
- [2] Rob Pike, Dave Presotto, Ken Thompson, Howard Trickey: *Plan 9 from Bell Labs*. UKUUG Summer Conference, 1990.
- [3] Gernot Heiser, Kevin Elphinstone: *L4 Microkernels: The Lessons from 20 Years of Research and Deployment*. ACM Trans. Comput. Syst. 34(1), 2016.
- [4] Norman Hardy: *The Confused Deputy (or why capabilities might have been invented)*. ACM SIGOPS Operating Systems Review 22(4), 1988.
- [5] Jerome H. Saltzer, Michael D. Schroeder: *The Protection of Information in Computer Systems*. Proceedings of the IEEE 63(9), 1975.
- [6] Martin Kleppmann: *Designing Data-Intensive Applications*. O'Reilly, 2017. (Kapitel über CAS und Replikation)
- [7] Paul E. McKenney: *Is Parallel Programming Hard, And, If So, What Can You Do About It?* kernel.org, 2022. (RCU, Epoch-basiertes Concurrency)
- [8] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu: *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise*. KDD-96, 1996.
- [9] UEFI Specification Version 2.10. Unified EFI Forum, 2022.
- [10] The Rust Reference: *The Rust Programming Language*. doc.rust-lang.org, 2024.
- [11] `uefi-rs` Crate: UEFI Bindings für Rust. github.com/rust-osdev/uefi-rs.
- [12] `postcard` Crate: A compact binary serialization format for `no_std`. github.com/jamesmunns/postcard.